Universidade Federal de Sergipe — **PROCC**
Projeto e Análise de Algoritmos · Período 2026.2
Módulo II — Otimização, Grafos e Processamento de Cadeias

Atividade 2 (AV2) — Relatório Técnico

Recuperação e seleção otimizada de contexto para IA generativa em materiais educacionais abertos

Programação dinâmica e mochila 0/1 para seleção de chunks sob restrição de tokens: um estudo comparativo com estratégia gulosa

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

Tema	8 — Materiais educacionais abertos
Corpus	*Pense Python* (tradução livre da 3ª ed. de *Think Python*)
Repositório	`https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_N`
Licença do código	MIT · Versão/tag: v0.1.0
Vídeo da atividade	*URL a preencher até 22/09/2026* (também em README, VIDEO.md e Classroom)

Documento gerado em 04/09/2026 · São Cristóvão, SE
Entrega dos artefatos até 23/09/2026 · Apresentação em 24/09/2026

Estado desta versão do relatório.

Este documento apresenta a evolução do protótipo desenvolvido na AV1, incorporando a seleção de contexto sob restrição de tokens, conforme a Trilha A da Atividade 2.

A abordagem principal será a programação dinâmica para o problema da mochila 0/1, comparada a uma estratégia gulosa por relevância/token.

A especificação formal, a implementação, a análise de corretude, a complexidade e os resultados experimentais serão documentados nas respectivas seções, conforme o desenvolvimento da atividade.

Sumário

1 Identificação da equipe **3**

2 Tema, corpus e fonte **3**

2.1 Continuidade da Atividade 1 4

3 Repositório, licença, versão e data de acesso **4**

4 Especificação do problema **5**

4.1 Definição do problema 5

4.2 Corpus e representação dos chunks 5

4.3 Função de relevância 6

4.4 Definição formal da entrada 7

4.5 Definição formal da saída 7

4.6 Função objetivo 8

4.7 Restrições do problema 8

4.8 Critério de corretude 9

4.9 Linha de base: estratégia gulosa 9

4.10 Casos de borda 10

4.11 Parâmetros experimentais preliminares 11

4.12 Decisões metodológicas 11

5 Representação dos dados e estratégia de chunking **12**

5.1 Normalização 12

5.2 Segmentação (chunking) 12

5.3 Representações computacionais 12

6 Algoritmos e pseudocódigo **13**

6.1 Programação dinâmica: mochila 0/1 . . . 13
6.2 Estratégia gulosa por relevância/token . . . 13
6.3 Justificativa de corretude . . . 13

7 Análise no modelo RAM **13**
7.1 Parâmetros . . . 13
7.2 Operações elementares . . . 13
7.3 Programação dinâmica: mochila 0/1 . . . 14
7.4 Estratégia gulosa por relevância/token . . . 14
7.5 Custo do pipeline de recuperação e seleção . . . 14

8 Melhor, pior e caso médio **14**

9 Recorrências **14**
9.1 Programação dinâmica: mochila 0/1 . . . 14
9.2 Relação com o custo computacional . . . 14

10 Metodologia experimental **14**
10.1 Configurações comparadas . . . 14
10.2 Cargas e repetições . . . 14
10.3 Métricas previstas . . . 14
10.4 Reprodutibilidade . . . 15

11 Resultados e gráficos **15**

12 Discussão de escalabilidade **15**

13 Relação com IA generativa e RAG **15**

14 Limitações e ameaças à validade **15**

15 Declaração de uso de IA generativa **16**

16 Contribuição individual **16**

17 Vídeo da atividade **17**

18 Referências **17**

1. Identificação da equipe

A Atividade 2 (AV2) é desenvolvida por seis discentes do Programa de Pós-Graduação em Ciência da Computação (PROCC/UFS), na disciplina Projeto e Análise de Algoritmos, período 2026.2.

As responsabilidades foram distribuídas conforme as etapas previstas no enunciado, contemplando especificação formal, implementação, análise assintótica, experimentação e relação com inteligência artificial generativa.

Tabela 1: Distribuição de responsabilidades na AV2.

Integrante	**Frente principal**
João Cosme Sena Sá	Parte B: algoritmo e corretude; Parte D: experimentos e comparação.
Eduardo Henrique do Lago Silva	Parte B: algoritmo e corretude; Parte C: análise assintótica; Parte D: experimentos e comparação.
Helena Carvalho Leal	Parte D: experimentos, comparação e métricas; Parte E: relação com IA generativa.
Hernandison da Silva Bispo	Parte A: especificação do problema; Parte E: relação com IA generativa.
Nadianne Maria dos Santos Galvão	Parte A: especificação formal do problema; Parte C: análise assintótica.
Rafael Takeguma Goto	Parte B: algoritmo e corretude; Parte E: relação com IA generativa.

A divisão apresentada estabelece as responsabilidades principais de cada integrante, permanecendo a revisão e a validação dos resultados como atividades coletivas.

> A divisão acima é a **divisão de frentes** acordada (`docs/CONTRIBUICOES.md`); a revisão de código, da prova e dos resultados é responsabilidade coletiva. A tabela de contribuição individual **com evidências** é mantida em `docs/CONTRIBUICOES.md` e resumida na Seção 16.

2. Tema, corpus e fonte

O tema é o de número 8 do enunciado, **materiais educacionais abertos**. O corpus é o livro *Pense Python*, tradução livre da 3ª edição de *Think Python*, de Allen B. Downey, traduzida por Rodrigo Castelan Carlson — conteúdo didático público de introdução à programação em Python, sem qualquer dado sensível, pessoal ou sigiloso.

Campo	**Registro**
Nome	*Pense Python* — tradução livre da 3ª ed. de *Think Python*
Autor / Tradução	Allen B. Downey / Rodrigo Castelan Carlson
Fonte	`https://rodrigocarlson.github.io/PensePython3ed/` ; `https://github.com/rodrigocarlson/PensePython3ed`
Commit de referência	`cfde2c450bf4f2ea65326da88e9968f400597e92` (branch `main`)
Recorte	21 notebooks Jupyter: prefácio, introdução e capítulos 1 a 19 (`capitulos/*.ipynb`)
Tamanho auditado	817.578 bytes · 2.789 células · ≈73.867 palavras
SHA-256 do recorte	`2010626258...5768dc5` (agregado; ver `data/corpus_manifest.csv`)
Idioma	Português brasileiro (com estrangeirismos técnicos do Python/Jupyter)
Licença do texto / código	CC BY-NC-SA 4.0 / MIT
Data de acesso	02/09/2026 (aquisição registrada); planejamento em 01/09/2026
Dados sensíveis	Nenhum — conteúdo didático público

2.1 Continuidade da Atividade 1

A Atividade 2 (AV2) dá continuidade ao protótipo de recuperação de contexto desenvolvido na Atividade 1 (AV1), preservando o corpus *Pense Python* e os mecanismos de recuperação lexical.

Na AV1, o sistema recuperava e ordenava fragmentos textuais (*chunks*) a partir de consultas em português, utilizando pontuações de relevância baseadas em frequência de termos e frequência inversa de documento.

Na AV2, incorpora-se uma etapa de otimização responsável por selecionar um subconjunto de chunks que maximize a relevância estimada, respeitando um orçamento máximo de tokens.

O problema será modelado como mochila 0/1, utilizando programação dinâmica como abordagem principal e uma estratégia gulosa por relevância/token como linha de base, conforme a formulação clássica de otimização combinatória apresentada por Kleinberg e Tardos [**?**].

> **Corpus não versionado, mas reprodutível.** Como a licença do texto é não comercial e com compartilhamento pelas mesmas condições (CC BY-NC-SA), o texto não é redistribuído no repositório (cujo código é MIT). O script `scripts/baixar_corpus.py` baixa o material no commit fixado e `data/corpus_manifest.csv` guarda o SHA-256 de cada notebook e um **hash agregado do recorte**, permitindo conferir que qualquer download reproduz o mesmo conjunto auditado. Trechos citados preservam atribuição ao autor e ao tradutor.

Consultas de recuperação. As consultas seguem o formato "tira-dúvidas" de um estudante sobre o livro, por exemplo: *"como funciona um laço `for` em Python?"*, *"o que é uma função recursiva?"* e *"como tratar exceções com `try` e `except`?"*.

O conjunto de 30 consultas, definido na AV1 antes da observação dos resultados, será reutilizado na AV2, preservando-se o arquivo `data/queries.csv`.

Para cada consulta, os chunks recuperados constituirão o conjunto de candidatos submetido às estratégias de programação dinâmica e gulosa, sob o mesmo orçamento de tokens.

3. Repositório, licença, versão e data de acesso

Item	Registro
URL do repositório	`https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo`
Licença do código	MIT (arquivo `LICENSE`)
Versão / tag	v0.1.0 (`CITATION.cff`, `date-released: 2026-09-03`)
Commit/tag/release do relatório	*a fixar na entrega final (tag da release de 23/09/2026)*
Data de acesso ao corpus	02/09/2026 (a reconfirmar no dia da aquisição)
Ambiente travado	`requirements.lock`: pytest 8.3.4, scikit-learn 1.6.1, matplotlib 3.10.0, numpy 2.2.2

O repositório não contém chaves, senhas, tokens, dados pessoais ou artefatos sem licença compatível. Arquivos grandes são controlados por `.gitignore`; as instruções de obtenção e reprodução estão no `README.md`.

4. Especificação do problema

4.1 Definição do problema

O problema investigado consiste em selecionar, entre os chunks recuperados pelo mecanismo lexical da AV1 para uma determinada consulta, um subconjunto que maximize a relevância estimada sem ultrapassar um orçamento máximo de tokens.

Cada chunk possui um custo, associado à sua quantidade de unidades de tokenização, e um valor, determinado pela pontuação de relevância atribuída durante a recuperação.

A seleção é modelada como um problema da mochila 0/1 (*0/1 Knapsack Problem*), no qual cada chunk pode ser incluído integralmente ou excluído da solução, sem possibilidade de fracionamento.

Essa modelagem constitui uma adaptação do problema clássico de otimização combinatória apresentado por Kleinberg e Tardos [**?**].

4.2 Corpus e representação dos chunks

O corpus utilizado corresponde ao livro *Pense Python*, tradução brasileira da terceira edição de *Think Python*, de Allen B. Downey, realizada por Rodrigo Castelan Carlson [**?**].

O recorte, preservado da AV1, compreende 21 notebooks Jupyter, incluindo materiais introdutórios e os capítulos 1 a 19.

O arquivo de dados analisado contém 2.737 chunks, sendo 1.405 de texto Markdown e 1.332 de código. Cada chunk possui identificador, conteúdo textual, ordem de origem e metadados de localização no corpus.

Para a AV2, será utilizada uma representação derivada dos candidatos recuperados, contendo os campos:

- `chunk_id`: identificador único;
- `texto`: conteúdo do fragmento;
- `tokens`: custo de seleção;
- `score_relevancia`: relevância estimada para a consulta;
- `source_order`: posição original;
- `origem`: procedência do fragmento.

O arquivo `data/processed/chunks.jsonl`, produzido na AV1, será preservado como fonte dos fragmentos.

Os registros originais possuem o campo `n_tokens`, que será inicialmente utilizado como custo de seleção, expresso em unidades lexicais do texto normalizado.

O script de preparação utiliza, por padrão, tamanho máximo de 256 unidades e sobreposição configurada de 32 unidades entre fragmentos.

Essa contagem constitui uma aproximação do custo de contexto, não correspondendo necessariamente à quantidade de tokens utilizada por um modelo de linguagem específico.

A representação da AV2 preservará os identificadores e metadados originais, acrescentando a pontuação de relevância calculada para cada consulta.

Dessa forma, ambos os algoritmos receberão os mesmos candidatos, custos e valores.

4.3 Função de relevância

A relevância lexical é calculada pelo mecanismo TF-IDF desenvolvido na AV1, com base na frequência de termos e na frequência inversa de documento, conceitos apresentados por Manning, Raghavan e Schütze [**?**].

Na AV2, preserva-se a mesma função de pontuação, de modo que as diferenças entre os algoritmos comparados estejam relacionadas à seleção dos chunks, e não a alterações no cálculo da relevância.

A frequência ponderada de um termo é definida por:

$$\mathrm{tf}(t,c) = \begin{cases} 1 + \ln f_{t,c}, & \text{se } f_{t,c} > 0, \\ 0, & \text{caso contrário.} \end{cases} \tag{1}$$

A frequência inversa de documento é calculada por:

$$\mathrm{idf}(t) = \ln\left(\frac{N+1}{\mathrm{df}(t)+1}\right) + 1, \tag{2}$$

em que N representa a quantidade total de chunks do corpus e $\mathrm{df}(t)$ corresponde ao número de chunks que contêm o termo t.

A pontuação de relevância para uma consulta q e um chunk c é:

$$\mathrm{score}(q,c) = \sum_{t \in \mathrm{Termos}(q)} qtf_t \cdot \mathrm{tf}(t,c) \cdot \mathrm{idf}(t). \tag{3}$$

Nessa expressão, qtf_t representa a frequência do termo t na consulta e $f_{t,c}$ sua frequência no chunk analisado. Termos ausentes do vocabulário não contribuem para a pontuação.

A formulação preserva o mecanismo de relevância implementado na AV1, mantendo a mesma pontuação para os algoritmos comparados.

Na AV2, essa pontuação será utilizada como valor do item no problema da mochila 0/1:

$$v_i = \mathrm{score}(q, c_i). \tag{4}$$

Dessa forma, a programação dinâmica e a estratégia gulosa utilizarão os mesmos valores de relevância para cada candidato.

Na recuperação da AV1, os chunks são ordenados por pontuação decrescente e, em caso de empate, pelo menor `source_order`. Essa regra será preservada na recuperação dos candidatos.

Ressalta-se que o TF-IDF constitui uma estimativa de relevância lexical. Portanto, a soma dos scores não representa necessariamente maior cobertura semântica, diversidade de informações ou ausência de redundância entre os chunks.

4.4 Definição formal da entrada

Para uma consulta q, considera-se o conjunto de chunks candidatos recuperados pelo mecanismo lexical da AV1, com pontuação de relevância positiva:

$$C_q = \{c_1, c_2, \ldots, c_n\}, \tag{5}$$

em que $n = |C_q|$ representa a quantidade de candidatos disponíveis para a seleção, e não o número total de chunks do corpus.

Cada candidato c_i possui:

- um custo $w_i \in \mathbb{N}^+$, correspondente à quantidade de unidades lexicais registrada em `n_tokens`;
- um valor $v_i \in \mathbb{R}_{\geq 0}$, correspondente à pontuação de relevância estimada para a consulta q.

Define-se $W \in \mathbb{N}_0$ como o orçamento máximo de unidades lexicais disponível para a seleção.

Assim, a entrada do problema é representada por:

$$I = (q, C_q, W), \tag{6}$$

em que cada candidato c_i está associado ao par (w_i, v_i).

Essa representação adapta a formulação clássica do problema da mochila 0/1, apresentada por Kleinberg e Tardos [**?**].

4.5 Definição formal da saída

A saída consiste em um subconjunto de chunks selecionados:

$$S \subseteq C_q, \tag{7}$$

no qual cada chunk pode ser incluído integralmente ou excluído, sem possibilidade de fracionamento ou seleção repetida. Uma solução é considerada viável quando respeita o orçamento máximo de unidades lexicais:

$$\sum_{c_i \in S} w_i \leq W. \tag{8}$$

Além do subconjunto selecionado, serão registrados o custo total da seleção e a relevância acumulada, definidos, respectivamente, por:

$$W_S = \sum_{c_i \in S} w_i, \tag{9}$$

$$V_S = \sum_{c_i \in S} v_i. \tag{10}$$

A saída deverá preservar os identificadores dos chunks selecionados, permitindo recuperar seus textos e metadados de origem. A exigência de maximizar a relevância acumulada será formalizada na função objetivo.

4.6 Função objetivo

O objetivo da seleção é maximizar a relevância lexical acumulada dos chunks escolhidos, considerando o conjunto de candidatos recuperados para uma consulta.

Para cada candidato c_i, define-se uma variável binária de decisão:

$$x_i = \begin{cases} 1, & \text{se o chunk } c_i \text{ for selecionado,} \\ 0, & \text{caso contrário.} \end{cases} \tag{11}$$

A função objetivo é definida por:

$$\max V_S = \max \sum_{i=1}^{n} v_i x_i. \tag{12}$$

Nessa expressão, n representa a quantidade de candidatos, v_i corresponde à relevância lexical do chunk c_i para a consulta e x_i indica sua inclusão na seleção.

A formulação segue o problema clássico da mochila 0/1, no qual se busca maximizar o valor total dos itens selecionados sob uma restrição de capacidade [**?**].

Neste trabalho, o valor de cada item corresponde ao score lexical calculado pelo mecanismo TF-IDF da AV1. Portanto, a maximização refere-se à relevância estimada pelo modelo adotado.

A função objetivo pressupõe valores aditivos: a relevância total é calculada pela soma dos scores individuais dos chunks selecionados.

Entretanto, essa soma não considera diretamente a redundância, as dependências ou a diversidade de informações entre os fragmentos.

Consequentemente, a solução ótima da mochila 0/1 não garante, isoladamente, maior cobertura semântica ou melhor qualidade das respostas geradas por um modelo de linguagem.

4.7 Restrições do problema

A seleção de contexto está sujeita a uma restrição de capacidade, determinada pelo orçamento máximo de unidades lexicais W.

A soma dos custos dos chunks selecionados não pode ultrapassar esse limite:

$$\sum_{i=1}^{n} w_i x_i \leq W, \tag{13}$$

em que w_i representa o custo do chunk c_i, obtido do campo `n_tokens` da AV1, e x_i indica sua seleção. Adicionalmente, cada candidato está sujeito à restrição de integralidade:

$$x_i \in \{0, 1\}, \quad i = 1, \ldots, n. \tag{14}$$

Assim, um chunk deve ser selecionado integralmente ou excluído, não podendo ser fracionado nem incluído mais de uma vez.

Considera-se também que os custos são inteiros positivos ($w_i \in \mathbb{N}^+$), os valores de relevância são reais não negativos e finitos ($v_i \in \mathbb{R}_{\geq 0}$), e o orçamento é um inteiro não negativo ($W \in \mathbb{N}_0$).

Essas condições caracterizam a formulação clássica da mochila 0/1, adaptada à seleção de contexto [**?**]. O orçamento W e os custos w_i serão expressos na mesma unidade, garantindo a consistência da restrição de capacidade.

4.8 Critério de corretude

Uma solução produzida pela programação dinâmica será considerada correta quando satisfizer simultaneamente duas propriedades: viabilidade e otimalidade.

A viabilidade exige que todos os chunks selecionados pertençam ao conjunto de candidatos C_q, que cada chunk seja incluído no máximo uma vez e que o orçamento de unidades lexicais seja respeitado:

$$S \subseteq C_q \quad \text{e} \quad \sum_{c_i \in S} w_i \leq W. \tag{15}$$

Define-se o conjunto de soluções viáveis por:

$$\mathcal{F} = \left\{ S' \subseteq C_q \;\middle|\; \sum_{c_i \in S'} w_i \leq W \right\}. \tag{16}$$

A otimalidade exige que não exista outro subconjunto viável com relevância acumulada superior à solução encontrada. Formalmente, uma solução ótima S^* deve satisfazer:

$$\sum_{c_i \in S^*} v_i \geq \sum_{c_i \in S'} v_i, \quad \forall S' \in \mathcal{F}. \tag{17}$$

Esse critério corresponde à otimalidade do problema clássico da mochila 0/1, fundamentado na programação dinâmica [**?**].

A programação dinâmica deverá satisfazer as condições de viabilidade e otimalidade. Por sua vez, a estratégia gulosa deverá garantir a viabilidade, mas não necessariamente a otimalidade global.

Ressalta-se que a corretude é estabelecida em relação à função objetivo e às restrições matemáticas definidas.

Portanto, uma solução ótima não garante, isoladamente, relevância semântica perfeita, ausência de redundância ou maior qualidade das respostas produzidas por um modelo de linguagem.

A demonstração formal da corretude do algoritmo implementado será apresentada na Parte B.

4.9 Linha de base: estratégia gulosa

Como linha de base (*baseline*), será utilizada uma estratégia gulosa que prioriza os chunks pela razão entre a relevância lexical estimada e o custo em unidades lexicais. Para cada candidato c_i, define-se:

$$r_i = \frac{v_i}{w_i}, \tag{18}$$

em que v_i representa a pontuação de relevância do candidato e w_i corresponde ao custo obtido do campo `n_tokens` da AV1.

Os candidatos serão considerados em ordem decrescente de r_i. Inicialmente, o orçamento restante será igual a W.

Cada chunk será selecionado integralmente caso seu custo não ultrapasse o orçamento restante. Após sua inclusão, o custo correspondente será descontado do orçamento disponível. Caso o candidato não caiba, será ignorado, e a avaliação continuará com o próximo item.

Em caso de empate na razão, será priorizado o candidato com maior relevância absoluta e, persistindo o empate, aquele com menor `source_order`, garantindo uma ordenação determinística.

Embora a estratégia gulosa apresente uma regra de escolha simples, ela não garante a solução ótima para a mochila 0/1, pois escolhas locais podem impedir combinações globalmente superiores [**?**]. A programação dinâmica e o baseline guloso receberão os mesmos candidatos, custos, pontuações de relevância e orçamento W.

A comparação considerará a relevância acumulada V_S, o custo total W_S, o tempo de seleção e o consumo aproximado de memória. A avaliação experimental dessas métricas será realizada posteriormente, na Parte D.

4.10 Casos de borda

Para assegurar a consistência da seleção de contexto, foram definidos casos de borda que deverão ser considerados na validação dos algoritmos.

A Tabela 2 apresenta os cenários e os comportamentos esperados.

Tabela 2: Casos de borda da seleção de contexto.

Caso	**Comportamento esperado**
Candidatos vazios	Se $C_q = \varnothing$, retornar $S = \varnothing$, com custo e relevância total iguais a zero.
Consulta sem correspondência	Caso nenhum chunk apresente relevância positiva, retornar o conjunto vazio.
Orçamento nulo	Se $W = 0$, retornar $S = \varnothing$, considerando que todos os custos são positivos.
Nenhum chunk cabe	Se $w_i > W$ para todos os candidatos, retornar o conjunto vazio.
Todos os chunks cabem	Se $\sum_{i=1}^{n} w_i \leq W$, selecionar todos os candidatos com relevância positiva.
Chunk maior que o orçamento	Candidatos com $w_i > W$ não podem integrar a solução, mas os demais continuam elegíveis.
Chunks duplicados	Identificadores repetidos serão tratados como entrada inválida. Chunks com textos idênticos, mas identificadores distintos, serão preservados inicialmente, e sua possível redundância será considerada na avaliação experimental.
Empate de relevância	Quando existirem soluções com o mesmo valor ótimo, priorizar aquela com menor custo total e, persistindo o empate, a menor sequência lexicográfica dos valores de `source_order` selecionados.
Empate na razão relevância/custo	No baseline guloso, priorizar maior relevância absoluta e, persistindo o empate, menor `source_order`.
Relevância zero	Chunks com $v_i = 0$ não acrescentam valor à função objetivo e podem ser descartados antes da seleção.

Os comportamentos descritos constituem requisitos esperados da implementação, e não resultados já observados.

Sua verificação será realizada posteriormente, por meio de testes automatizados e experimentos específicos na Parte D.

Para a programação dinâmica, o desempate entre soluções ótimas seguirá, nesta ordem, a maior relevância acumulada, o menor custo total e a menor sequência lexicográfica dos valores de `source_order`,

previamente ordenados em ordem crescente.

4.11 Parâmetros experimentais preliminares

Os experimentos serão planejados para comparar a programação dinâmica e a estratégia gulosa sob condições equivalentes, reutilizando o corpus, as consultas e o mecanismo de recuperação da AV1.

A Tabela 3 apresenta os parâmetros experimentais preliminares.

Tabela 3: Parâmetros experimentais preliminares da AV2.

Parâmetro	**Definição preliminar**
Corpus	2.737 chunks do livro *Pense Python*, reutilizados da AV1.
Consultas	30 consultas previamente definidas em `data/queries.csv`.
Recuperação	Configuração C3 da AV1: busca linear com ordenação por Merge Sort.
Orçamento (W)	256, 512 e 1.024 unidades lexicais, sujeitos à validação.
Quantidade de candidatos (n)	10, 30 e 50 candidatos por consulta, limitados aos resultados com relevância positiva.
Repetições	No mínimo três execuções por cenário.
Algoritmo principal	Mochila 0/1 com programação dinâmica.
Linha de base	Estratégia gulosa por relevância/custo.
Métricas de qualidade	Relevância acumulada (V_S), custo total (W_S), taxa de utilização do orçamento, redundância e diferença de relevância entre as estratégias.
Métricas computacionais	Tempo de seleção, tempo total do fluxo e consumo aproximado de memória.

Para assegurar a comparabilidade, ambos os algoritmos receberão os mesmos candidatos, custos, pontuações de relevância e orçamento.

O mecanismo de recuperação permanecerá fixo durante as comparações, de modo que as diferenças observadas possam ser atribuídas às estratégias de seleção. Os valores apresentados constituem um planejamento inicial. A quantidade efetiva de candidatos, os orçamentos e as repetições serão confirmados antes da execução dos experimentos.

O custo dos chunks será inicialmente obtido do campo `n_tokens`, interpretado como contagem de unidades lexicais, e não como tokenização exata de um LLM. A implementação dos algoritmos, a execução dos experimentos, a coleta das métricas e a análise dos resultados serão realizadas nas Partes B e D, respectivamente.

4.12 Decisões metodológicas

Para preservar a continuidade da AV1, será utilizada a configuração C3, composta por busca linear e Merge Sort, como mecanismo fixo de recuperação dos candidatos.

O custo w_i será definido inicialmente pelo campo `n_tokens` da AV1, correspondente à contagem de unidades lexicais do texto normalizado, separadas por espaços em branco. Essa medida será tratada como uma aproximação do custo de contexto, não equivalendo necessariamente à quantidade de tokens processados por um modelo de linguagem.

Em caso de múltiplas soluções com a mesma relevância acumulada máxima na programação dinâmica, será priorizada aquela com menor custo total.

Persistindo o empate, será selecionado o subconjunto cuja sequência crescente de valores `source_order` seja lexicograficamente menor, garantindo uma regra determinística. A programação dinâmica e

a estratégia gulosa receberão os mesmos conjuntos de candidatos, valores de relevância, custos e orçamentos.

Essa padronização busca assegurar que as diferenças observadas nos experimentos sejam atribuídas às estratégias de seleção, e não a alterações nos dados de entrada.

5. Representação dos dados e estratégia de chunking

5.1 Normalização

A AV2 preserva o procedimento de normalização desenvolvido na AV1, mantendo as seguintes regras:

- **Células Markdown:** normalização NFKC, conversão para minúsculas, separação controlada da pontuação e compactação de espaços, preservando os acentos.
- **Células de código:** preservação do código-fonte original, incluindo a indentação, com representação lexical separada.
- **Stopwords:** não são removidas na configuração principal, preservando termos funcionais relevantes para as consultas.
- **Saídas de execução:** são descartadas durante a preparação do corpus.

5.2 Segmentação (chunking)

A segmentação preserva a hierarquia do livro, considerando notebook, capítulo e seção, sem combinar células Markdown e código em um mesmo chunk.

O procedimento de preparação da AV1 utiliza, por padrão, tamanho máximo de 256 unidades lexicais e sobreposição configurada de 32 unidades, produzindo 2.737 chunks.

As configurações 128/16 e 512/64 foram consideradas na análise de sensibilidade da AV1, não sendo inicialmente adotadas como configurações principais da AV2.

A contagem registrada no campo `n_tokens` será utilizada inicialmente como aproximação do custo de seleção. Essa medida não equivale necessariamente à tokenização de um modelo de linguagem.

5.3 Representações computacionais

A AV2 reutiliza a representação lexical e os mecanismos de pontuação desenvolvidos na AV1, incorporando uma estrutura específica para a seleção de contexto.

- **TF-IDF:** representa a relevância lexical dos chunks em relação à consulta, fornecendo o valor v_i de cada candidato.
- **Custo de seleção:** cada chunk possui um custo w_i, associado à contagem de unidades lexicais registrada no corpus.
- **Representação da mochila 0/1:** cada candidato é representado pelo par (w_i, v_i), acompanhado de seu identificador e metadados de origem.
- **Orçamento:** o limite W estabelece a capacidade máxima disponível para selecionar os chunks.

A recuperação dos candidatos será realizada pela configuração C3 da AV1, composta por busca linear e Merge Sort, mantendo fixos os critérios de pontuação e ordenação.

Limitações da representação.

O corpus contém termos técnicos em inglês, como *loop* e *string*, que podem influenciar a relevância lexical.

Além disso, a utilização de um único livro introdutório limita a generalização dos resultados para outros materiais educacionais.

A função objetivo considera a soma das relevâncias individuais, não modelando diretamente redundância ou cobertura semântica entre os chunks.

6. Algoritmos e pseudocódigo

A AV2 reutiliza o mecanismo de recuperação lexical da AV1, adotando a configuração C3, composta por busca linear e Merge Sort.

A principal alteração consiste na inclusão de uma etapa de seleção de contexto sob restrição de tokens, modelada como mochila 0/1.

Serão consideradas duas abordagens: programação dinâmica, como algoritmo principal, e uma estratégia gulosa por relevância/token, utilizada como linha de base.

6.1 Programação dinâmica: mochila 0/1

6.2 Estratégia gulosa por relevância/token

6.3 Justificativa de corretude

7. Análise no modelo RAM

7.1 Parâmetros

Tabela 4: Parâmetros da análise de complexidade.

Símbolo	Significado
N	Número total de chunks do corpus.
n	Número de chunks candidatos para uma consulta.
W	Orçamento máximo de unidades de tokenização.
w_i	Custo do candidato i.
v_i	Valor de relevância do candidato i.
m	Número de termos da consulta.
r	Número de repetições experimentais.

7.2 Operações elementares

A análise considerará operações elementares do modelo RAM, incluindo comparações, atribuições, acessos a estruturas de dados e operações aritméticas de custo constante.

7.3 Programação dinâmica: mochila 0/1

7.4 Estratégia gulosa por relevância/token

7.5 Custo do pipeline de recuperação e seleção

8. Melhor, pior e caso médio

9. Recorrências

A análise das recorrências será concentrada na programação dinâmica aplicada ao problema da mochila 0/1, considerando a quantidade de chunks candidatos e o orçamento disponível.

9.1 Programação dinâmica: mochila 0/1

9.2 Relação com o custo computacional

10. Metodologia experimental

10.1 Configurações comparadas

A avaliação experimental comparará duas estratégias de seleção de contexto, utilizando o mesmo conjunto de candidatos recuperados pela configuração C3 da AV1 (busca linear e Merge Sort).

Tabela 5: Estratégias previstas para comparação.

Estratégia	**Descrição**
Programação dinâmica	Seleção ótima de chunks por mochila 0/1, considerando relevância e orçamento.
Estratégia gulosa	Seleção sequencial de chunks por razão relevância/token, respeitando o orçamento.

10.2 Cargas e repetições

O planejamento inicial considera 30 consultas herdadas da AV1, com orçamentos de 256, 512 e 1.024 unidades de tokenização.

Serão considerados conjuntos de 10, 30 e 50 candidatos, respeitando a quantidade disponível de resultados com relevância positiva.

Cada cenário será executado pelo menos três vezes, quando o ambiente permitir.

Os valores poderão ser ajustados após a verificação da viabilidade experimental.

10.3 Métricas previstas

- Relevância total dos chunks selecionados.
- Quantidade de unidades de tokenização utilizadas e respeito ao orçamento.
- Taxa de utilização do orçamento.

- Redundância entre os chunks selecionados.
- Tempo de seleção e tempo total do fluxo.
- Consumo aproximado de memória.

10.4 Reprodutibilidade

Os experimentos deverão registrar os parâmetros utilizados, a versão do corpus, a configuração de recuperação, as dependências, o ambiente computacional e o commit correspondente à AV2.

A recuperação será mantida fixa, garantindo que ambos os algoritmos recebam os mesmos candidatos e valores.

11. Resultados e gráficos

12. Discussão de escalabilidade

13. Relação com IA generativa e RAG

A seleção de contexto proposta na AV2 pode ser integrada à etapa de preparação de informações em sistemas de geração aumentada por recuperação (*Retrieval-Augmented Generation* — RAG).

Nesse cenário, os chunks recuperados da AV1 constituem candidatos à composição do contexto fornecido ao modelo de linguagem.

A AV2 acrescenta uma etapa de seleção sob orçamento, buscando maximizar a relevância lexical acumulada sem ultrapassar a capacidade estabelecida para os fragmentos.

A seleção ótima no modelo da mochila 0/1 não garante, isoladamente, maior qualidade das respostas, pois redundância, diversidade e relevância semântica não são diretamente representadas pela função objetivo.

14. Limitações e ameaças à validade

As principais limitações previstas para a abordagem são:

- **Relevância lexical:** o score TF-IDF depende da correspondência entre os termos da consulta e dos chunks, não representando diretamente similaridade semântica.
- **Aditividade:** a função objetivo soma relevâncias individuais, sem penalizar automaticamente informações redundantes.
- **Tokenização:** o custo inicial utiliza unidades lexicais registradas na AV1, que não equivalem necessariamente aos tokens de um LLM.
- **Corpus:** o estudo utiliza um único livro introdutório de programação, limitando a generalização dos resultados.
- **Recuperação:** a etapa de seleção depende dos candidatos produzidos pelo mecanismo lexical da AV1. Chunks relevantes não recuperados não podem ser selecionados posteriormente.
- **Validade experimental:** as conclusões dependerão dos orçamentos, da quantidade de candidatos, das consultas e do ambiente computacional utilizados.

A influência dessas limitações será discutida à luz dos resultados experimentais obtidos nas etapas posteriores.

15. Declaração de uso de IA generativa

A declaração detalhada é mantida **durante** o trabalho em `docs/USO_DE_IA.md`, com ferramenta, modelo, finalidade, até cinco prompts reais, sugestões aproveitadas, sugestões corrigidas ou rejeitadas, erros identificados e a verificação aplicada. Resumo até esta versão:

Ferramenta / modelo	**Finalidade**	**Verificação exigida**
Assistente de IA generativa (registrar o modelo exibido na interface)	Apoio à estruturação e redação deste relatório e à formatação da prova e dos pseudocódigos.	Toda saída de IA foi conferida quanto à corretude, complexidade, casos de borda, estabilidade de ordenação, desempate e coerência entre análise e experimento. Código, prova ou análise não verificados não foram incorporados ao trabalho.

> **Compromisso de uso crítico.** A prova da Seção 7 foi validada por testes de ordenação e desempate; os pseudocódigos foram confrontados com a implementação e os testes unitários; e as análises da Seção 12 foram verificadas por meio das execuções experimentais.

16. Contribuição individual

O enunciado pede contribuição **com evidência**, não cargo. A tabela final, com commits, arquivos e execuções, é mantida em `docs/CONTRIBUICOES.md`. Todos participam da revisão coletiva e da apresentação e do vídeo.

Integrante	**Frente**	**Evidências (a consolidar)**
João Cosme Sena Sá	Índice invertido / busca binária (C2); dados de teste, pacote e protocolo	`indice.py`, `busca_binaria.py`, `modelos.py`, fixtures
Eduardo Henrique do Lago Silva	Experimentos / figuras / baseline sklearn (C4)	`rodar_experimentos.py`, `gerar_figuras.py`, análise de C4
Helena Carvalho Leal	Consultas / qrels / métricas / IA / limitações	`queries.csv`, `qrels.csv`, `metricas.py`
Hernandison da Silva Bispo	Ingestão / normalização / fragmentação / relatório	`preprocessing.py`, `fragmentacao.py`, `scripts/`, `relatorio/`
Nadianne Maria dos Santos Galvão	Corretude / RAM / complexidade / recorrências	`CORRETUDE.md`, `test_sort_key.py`, Seções 7–10
Rafael Takeguma Goto	Busca linear / Insertion / Merge / pontuação (C1 e 3)	`linear_search.py`, `insertion_sort.py`, `merge_sort.py`, `relevance.py`

17. Vídeo da atividade

URL: *a preencher até 22/09/2026.* O vídeo terá até 10 minutos, com participação de todos e identificação clara de cada contribuição. A mesma URL constará no `README.md` (seção "Vídeo da atividade"), na capa deste relatório, no `VIDEO.md` (com data de gravação e participantes) e no Google Classroom quando solicitado. O vídeo apresentará: identificação da equipe, tema e corpus; o problema de recuperação; os algoritmos; uma demonstração breve; a corretude; a análise assintótica e as recorrências; os resultados e o gráfico principal; a relação com RAG; e as limitações e decisões de projeto.

18. Referências

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.
2. KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Pearson/Addison-Wesley, 2006.
3. SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Springer, 2020.
4. SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.
5. MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
6. LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020. Disponível em: `https://arxiv.org/abs/2005.11401`.
7. DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly Media, 2024. Tradução brasileira: *Pense Python*, por Rodrigo Castelan Carlson. Disponível em: `https://rodrigocarlson.github.io/PensePython3ed/`.
8. PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.